PROMETEO

Proyecto Intermodular 1º DAW

Participantes

Jheremi Loachamin Lema

Juan Camilo Montero

Índice

1. **Introducción**
   1.1Objetivos del proyecto
   1.2 Justificación
2. **Descripción del proyecto**
   2.1 Idea general del sistema
   2.2 Funcionalidades principales
   2.3 Alcance del proyecto
3. **Tecnologías utilizadas**
   3.1 Lenguajes de programación
   3.2 Herramientas de desarrollo
   3.3 Librerías o Frameworks utilizados
4. **Análisis del sistema**
   4.1 Identificación de necesidades
   4.2 Requisitos funcionales
   4.3 Requisitos no funcionales
5. **Diseño del sistema**
   5.1 Arquitectura del proyecto
   5.2 Diseño de la base de datos
   5.3 Estructura del proyecto
6. **Desarrollo del proyecto**
   6.2 Implementación de la interfaz web
   6.3 Integración entre frontend y backend
7. **Pruebas y validación**
   7.1 Pruebas realizadas
   7.2 Resultados obtenidos
8. **Conclusiones**
   8.1 Resultados del proyecto
   8.2 Mejoras futuras
9. **Bibliografía y recursos**
10. **Anexos**

1. Introducción

En la actualidad, la digitalización de los negocios se ha convertido en un elemento fundamental para mejorar la organización y la gestión de los servicios que ofrecen, muchos establecimientos deciden invertir sus recursos en sistemas informáticos para gestionar los servicios de una manera más correcta, por ejemplo, las barberías. Utilizan sistemas informáticos para gestionar reservas, clientes y servicios de forma más eficiente.

El presente proyecto, denominado BarberAPI, consiste en el desarrollo de una aplicación web que permite gestionar los principales elementos de una Barberia. A través de este sistema es posible registrar clientes, administrar citas y organizar los servicios ofrecidos por la Barberia.

Este proyecto se ha desarrollado con el objetivo de aplicar los conocimientos adquiridos en los distintos módulos del ciclo formativo, integrando conceptos de programación, lenguaje de marcas, gestión de datos, diferentes entornos o sistemas donde se pueda visualizar la aplicación y la presentación personal del proyecto y los contribuyentes.

1.1 Objetivos del proyecto

El objetivo de este proyecto es desarrollar una web para la gestión de una Barberia, permitiendo administrar información relacionada con **clientes**, **citas** y **servicios**.

Entre los *objetivos específicos* se encuentran:

- *Diseñar e implementar una web funcional para gestionar los servicios de una Barberia*.
- *Permitir la gestión de clientes y reservas de citas*.
- *Aplicar conocimientos de programación, desarrollo web, bases de datos, sistemas informáticos y documentación*.
- *Organizar el proyecto siguiente una estructura clara y modular*.
- *Utilizar herramientas de desarrollo y control versiones como Git y GitHub*.

- *Simular el funcionamiento de un sistema real de gestión de barberías*.

1.2 Justificación

La realización de este proyecto permite integrar y poner en práctica los conocimientos adquiridos durante el ciclo formativo. El desarrollo de BarberAPI tiene como finalidad crear una web que simule un sistema real utilizado con muchos negocios para gestionar sus servicios y clientes.

Además, este proyecto permite trabajar diferentes aspectos del desarrollo de software, como el diseño web, la organización de código, la documentación del proyecto y la utilización de herramientas colaborativas como GitHub. Cabe recalcar que BarberAPI será de código abierto para toda la comunidad que desee colaborar con un mejor desarrollo y proyección.

De esta forma, el proyecto no solo cumple un objetivo académico, sino que también representa una experiencia práctica que ayuda a comprender como se desarrollan aplicaciones reales en el ámbito profesional.

2. Descripción del Proyecto

2.1 Idea General del Sistema

El sistema "BarberAPI" nace como una plataforma digital integra que tiene como objetivo optimizar y modernizar la gestión diaria de una barbería. La idea principal es garantizar un entorno unificado donde se encuentren las necesidades de los clientes y la administración interna del negocio.

El proyecto se divide en dos grandes bloques:

- **Portal WEB Orientado al Cliente**: Una interfaz pública, accesible e intuitiva, cuya funcionalidad es ser una herramienta digital de la barbería. A través del portal, los usuarios podrán informarse sobre los servicios que se ofrecen en BarberAPI, conocer tarifas, visualizar la información de contacto y, principalmente, realizar reservas de citas de forma autónoma, sin depender de llamadas telefónicas o tiempos de espera.
- **Sistema de Gestión Interna (BackEnd)**: El núcleo lógico de la aplicación, encargado de procesar la información de manera segura y eficiente. Este sistema interactuará de forma directa con una base de

datos centralizada, permitiendo a los administradores del negocio (los barberos o gerentes) tener un control total sobre:

- **Gestión de Citas**: Visualizar, confirmar, modificar o cancelar las reservas realizadas por los clientes.

Por consiguiente, la idea general es crear una herramienta que automatice las tareas rutinarias, mejore la experiencia con el usuario, actualice aun sistema informático que organice las operaciones diarias de la barbería.

2.2 Funcionalidades Principales

El sistema de BarberAPI está diseñado para poder ofrecer un conjunto de herramientas que cubran tanto las necesidades de los usuarios finales como la del personal de la barbería, cuyas funcionalidades del sistema se dividen en dos módulos principales según el rol del usuario.

A. Modulo del Cliente (Interfaz Pública)

- **Catálogo de servicios:** Visualización de los distintos servicios que ofrece la barbería (corte de pelo, arreglo de barba, tintes, etc.), incluyendo su descripción, duración estimada y precio.
- **Gestión de reservas (CRUD):** Permite al cliente seleccionar un servicio, elegir al profesional (barbero) de su preferencia, y escoger una fecha y hora disponibles para agendar su cita, en caso de no poder cambiar horario, cancelar cita o buscar su cita.
- **Información corporativa:** Acceso a datos de contacto, ubicación del local, horarios de apertura y enlaces a redes sociales.

2.3 Alcance del proyecto

El proyecto BarberAPI abarca el diseño, desarrollo y documentación de un sistema funcional que simula la gestión de una barbería real. El desarrollo se limitará a las competencias adquiridas durante el primer año del ciclo de Desarrollo de Aplicaciones Web (DAW).

Alcance dentro del proyecto:

- **Diseño e implementación de la Base de Datos:** Creación de un modelo relacional en SQL que almacene de forma estructurada la información de clientes, empleados (barberos), catálogo de servicios y registro de reservas.

- **Desarrollo del Frontend:** Creación de una interfaz web corporativa estática utilizando HTML semántico y CSS, asegurando una navegación lógica y un diseño *responsive* básico (adaptación a dispositivos móviles).
- **Desarrollo del Backend:** Programación de la lógica de negocio mediante una aplicación en Java, aplicando los principios de la Programación Orientada a Objetos (POO).
- **Conectividad a Datos:** Integración del backend con la base de datos a través de JDBC para permitir operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre los registros del sistema.
- **Documentación Técnica y Despliegue:** Elaboración de un informe técnico sobre los requisitos del sistema y el entorno de ejecución necesario para hacer funcionar la aplicación de manera local.

Fuera del alcance del proyecto (Limitaciones):

Debido a que el sistema tiene un enfoque puramente académico y demostrativo para el primer año del ciclo formativo.

3. Tecnologías utilizadas

Para el desarrollo del sistema BarberAPI, se ha seleccionado un conjunto de tecnologías empleadas como requerimientos técnicos del primer curso del ciclo de Desarrollo de Aplicaciones Web (DAW).

3.1 Lenguajes de programación

- **Java:** Lenguaje principal utilizado para el desarrollo del *Backend* y la lógica de negocio de la aplicación. Se ha aplicado el paradigma de Programación Orientada a Objetos (POO) para estructurar el código de manera eficiente, segura y escalable.
- **SQL (Structured Query Language):** Diseño, creación y manipulación de la base de datos relacional del sistema. Mediante query ´s SQL que definen las tablas de clientes, servicios y reservas, así como la inserción de datos de prueba.
- **HTML5 y CSS3:** Lenguajes de marcas y estilos empleados para la maquetación y el diseño visual de la interfaz web del cliente. Garantizan una estructura semántica correcta y un diseño coherente y adaptable.

3.2 Herramientas de desarrollo

- **Git y GitHub:** Sistema de control de versiones distribuido y plataforma de alojamiento de repositorios, respectivamente. Utilizados para la gestión del código fuente, el trabajo colaborativo en ramas (*branches*), la integración de cambios (*Pull Requests*) y el mantenimiento de un historial de *commits* coherente.
- **Entorno de Desarrollo Integrado (IDE):** *IntelliJ IDEA y Visual Studio Code*. Utilizado para la escritura, depuración y compilación del código Java y el desarrollo web.
- **Sistema Gestor de Bases de Datos (SGBD):** *MySQL Workbench*. Empleado para administrar y visualizar el modelo relacional del proyecto.

3.3 Librerías o frameworks utilizados

La principal tecnología de integración utilizada es:

- **JDBC (Java Database Connectivity):** API de Java requerida de forma obligatoria para establecer la conexión entre la aplicación *Backend* y la base de datos relacional. Permite ejecutar las sentencias SQL (operaciones CRUD) directamente desde el código Java.

4. Análisis del sistema

Este apartado detalla el proceso de estudio previo a la programación, donde se han identificado las posibles debilidades encontradas en los modelos de gestión tradicional de una barbería y se han establecido las soluciones para que el sistema BarberAPI pueda lograr sus objetivos.

4.1 Identificación de necesidades

El diseño de BarberAPI surge de la observación de los problemas comunes a los que se enfrenta una barbería que gestiona sus citas de forma manual o analógica (por ejemplo, usando una agenda de papel o recibiendo reservas exclusivamente por teléfono o mensajes):

- **Dependencia del horario comercial:** Los clientes solo pueden reservar cuando el local está abierto y alguien puede atender el teléfono, lo que limita la captación de reservas.
- **Sobrecarga de trabajo:** El personal debe interrumpir constantemente sus servicios para atender llamadas y apuntar citas.

- **Errores humanos:** Al anotar a mano, es frecuente que haya solapamientos de horarios, pérdida de datos de contacto o confusiones sobre el servicio solicitado.
- **Falta de autonomía del cliente:** El usuario no tiene una forma rápida de consultar sus citas programadas o cancelarlas sin tener que contactar directamente con el local.

BarberAPI nace para digitalizar este proceso, centralizando la información en una base de datos y dándole al cliente la capacidad de autogestionar sus reservas en cualquier momento.

4.2 Requisitos funcionales

Los requisitos funcionales describen las acciones específicas que el sistema (la aplicación Java conectada a la base de datos) debe ser capaz de realizar. Basado en el alcance definido, el sistema debe cumplir con los siguientes puntos:

- **RF-01 (Gestión de Servicios):** El sistema debe permitir consultar el catálogo de servicios disponibles (corte, barba, tintes, etc.) almacenados en la base de datos, mostrando su precio y duración.
- **RF-02 (Registro de Clientes):** El sistema debe permitir almacenar y consultar los datos de los clientes (nombre, teléfono, correo) para asociarlos a sus respectivas reservas.
- **RF-03 (Creación de Reservas):** El sistema debe permitir a un cliente registrar una nueva cita, seleccionando el servicio deseado, la fecha y la hora (Operación *Create* del CRUD).
- **RF-04 (Consulta de Reservas):** El sistema debe permitir a un cliente buscar y visualizar los detalles de su cita programada (Operación *Read* del CRUD).
- **RF-05 (Modificación de Reservas):** El sistema debe permitir actualizar una reserva existente, cambiando el horario o el servicio seleccionado (Operación *Update* del CRUD).
- **RF-06 (Cancelación de Reservas):** El sistema debe permitir eliminar una reserva programada si el cliente decide cancelarla (Operación *Delete* del CRUD).

4.3 Requisitos no funcionales

Los requisitos no funcionales definen los atributos de calidad, rendimiento y restricciones del entorno del sistema:

- **RNF-01 (Usabilidad Web):** La interfaz web corporativa debe ser intuitiva, con una navegación clara y un diseño *responsive* básico que permita su correcta visualización tanto en monitores de escritorio como en dispositivos móviles.
- **RNF-02 (Persistencia de Datos):** Toda la información debe almacenarse de forma persistente y estructurada en una base de datos relacional MySQL.
- **RNF-03 (Arquitectura de Software):** El código backend (Java) debe seguir los principios de la Programación Orientada a Objetos, garantizando la modularidad y separación de responsabilidades (por ejemplo, separando la lógica de conexión a la base de datos del resto de la aplicación).
- **RNF-04 (Entorno de Ejecución):** El sistema debe estar preparado para ejecutarse y probarse en un entorno local, requiriendo un SGBD activo y la configuración correcta del Driver JDBC.

5. Diseño del sistema

En esta fase se definen los cimientos técnicos de BarberAPI, estableciendo la arquitectura de software elegida, el modelo de datos que soportará la información y la organización lógica de los archivos del proyecto.

5.1 Arquitectura del proyecto

El sistema BarberAPI se ha diseñado siguiendo una arquitectura monolítica basada en capas, lo que facilita la separación de responsabilidades y el mantenimiento del código, adaptándose a los estándares exigidos en el primer curso de DAW. Las capas principales son:

- **Capa de Presentación (Frontend):** Compuesta por los archivos estáticos HTML5 y CSS3. Es la interfaz gráfica mediante la cual el cliente interactúa con el catálogo de servicios y la información de la barbería.
- **Capa de Lógica de Negocio (Backend):** Desarrollada en Java. Contiene las clases y métodos que procesan las reglas del negocio, como la validación de datos al crear una reserva o la gestión de los menús de la aplicación.
- **Capa de Acceso a Datos:** Actúa como puente entre la lógica de negocio y la base de datos. Utiliza la API JDBC de Java para enviar sentencias SQL y recuperar los resultados.

- **Capa de Persistencia (Base de Datos):** El servidor MySQL donde reside la información estructurada de la aplicación, garantizando que los datos de citas y clientes no se pierdan al cerrar el programa.

5.2 Diseño de la base de datos

Para el almacenamiento de la información, se ha optado por una base de datos relacional. El diseño se compone del correspondiente Diagrama Entidad-Relación (E/R) y su paso al modelo relacional.

Entidades principales y sus atributos:

1. **Tabla clientes**: Almacena la información de los usuarios que realizan las reservas.
    a. id_cliente (Primary Key, Autoincremental): Identificador único del cliente.
    b. nombre: Nombre del cliente.
    c. apellidos: Apellidos del cliente.
    d. telefono: Teléfono de contacto.
    e. email: Correo electrónico.
2. **Tabla servicios**: Contiene el catálogo de tratamientos que ofrece la barbería.
    a. id_servicio (Primary Key, Autoincremental): Identificador único del servicio.
    b. nombre_servicio: Nombre (ej. Corte clásico, Arreglo de barba).
    c. descripcion: Breve detalle de lo que incluye.
    d. precio: Coste del servicio (Decimal).
    e. duracion_minutos: Tiempo estimado que tarda el barbero en realizarlo.
3. **Tabla reservas**: Es la tabla transaccional que une a los clientes con los servicios elegidos en una fecha concreta.
    a. id_reserva (Primary Key, Autoincremental): Identificador único de la cita.
    b. id_cliente (Foreign Key): Relaciona la cita con el cliente que la solicita.
    c. id_servicio (Foreign Key): Relaciona la cita con el tratamiento elegido.
    d. fecha_hora: Fecha y hora exacta de la cita (Datatype: DATETIME).
    e. estado: Situación de la reserva (ej. 'Pendiente', 'Completada', 'Cancelada').

5.3 Estructura del proyecto

Para garantizar un entorno de desarrollo organizado y colaborativo mediante Git/GitHub, el repositorio del proyecto se ha estructurado en los siguientes directorios principales:

- `/web`: Contiene todos los archivos de la interfaz de usuario (Lenguajes de Marcas). Incluye los ficheros `.html`, las hojas de estilo `.css` y la carpeta `/assets` para las imágenes del negocio.
- `/src`: Directorio raíz del código fuente en Java. En su interior, el código se divide en paquetes lógicos.
- `/sql`: Almacena los scripts de base de datos (`creacion_tablas.sql`, `insercion_datos.sql`) necesarios para recrear el entorno de persistencia en cualquier equipo.
- `/docs`: Destinado a la documentación del proyecto, incluyendo este informe técnico, diagramas, manuales de usuario y los documentos del perfil profesional requeridos para el módulo de Empleabilidad.

6. Desarrollo del proyecto

Este apartado describe el proceso de construcción técnica de BarberAPI, detallando cómo se ha programado la interfaz visual, la lógica de negocio y cómo interactúan los diferentes componentes del sistema para cumplir con los requisitos establecidos.

6.1 Implementación de la interfaz web (Frontend)

La interfaz pública del cliente se ha desarrollado buscando un equilibrio entre la estética visual, la usabilidad y el cumplimiento estricto de los estándares web.

- **Estructura con HTML5:** Se han creado las diferentes vistas de la aplicación utilizando etiquetas semánticas (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) para garantizar que el código sea accesible y fácil de leer. La web incluye la página de inicio, el catálogo de servicios (tablas o tarjetas) y los formularios de registro y reserva.
- **Diseño visual con CSS3:** Se ha definido una hoja de estilos centralizada que aporta la identidad corporativa a BarberAPI. Se han utilizado variables CSS para los colores principales (tonos propios de una barbería clásica o moderna) y se ha aplicado un modelo de cajas (*Flexbox* o *Grid*) para alinear los elementos.

- **Adaptabilidad (Responsive Design):** Mediante el uso de *Media Queries* en CSS, se ha garantizado que la web ofrezca una experiencia de navegación correcta tanto en resoluciones de escritorio como en dispositivos móviles, reestructurando los menús y apilando las columnas cuando es necesario.

6.2 Implementación de la Lógica Backend

El "Backend" o motor de la aplicación se ha construido utilizando Java Core, aplicando los principios de la Programación Orientada a Objetos (POO). El desarrollo se ha estructurado en las siguientes fases:

- **Creación del Modelo de Datos (Clases):** Se han programado las clases Java correspondientes a las entidades de la base de datos (`Cliente`, `Servicio`, `Reserva`). Estas clases utilizan encapsulamiento (atributos privados y métodos *getters/setters*) para proteger la información en memoria.
- **Desarrollo del DAO (Data Access Object):** Se ha implementado un patrón de diseño básico para separar la lógica de conexión a la base de datos del resto del programa. En estas clases se concentran las operaciones CRUD.
- **Conexión mediante JDBC:** Se ha utilizado el *driver* JDBC de MySQL para establecer el puente entre la aplicación Java y la base de datos. A través de la clase `Connection`, el sistema envía las sentencias SQL (previamente diseñadas con `INSERT`, `UPDATE`, `DELETE` y `SELECT`) y procesa los `ResultSet` devueltos para transformarlos en objetos Java utilizables por el programa.

6.3 Integración entre Frontend y Backend

Para el nivel del primer curso de DAW, la integración de los sistemas se enfoca en la coherencia de los datos y el funcionamiento unificado del proyecto global.

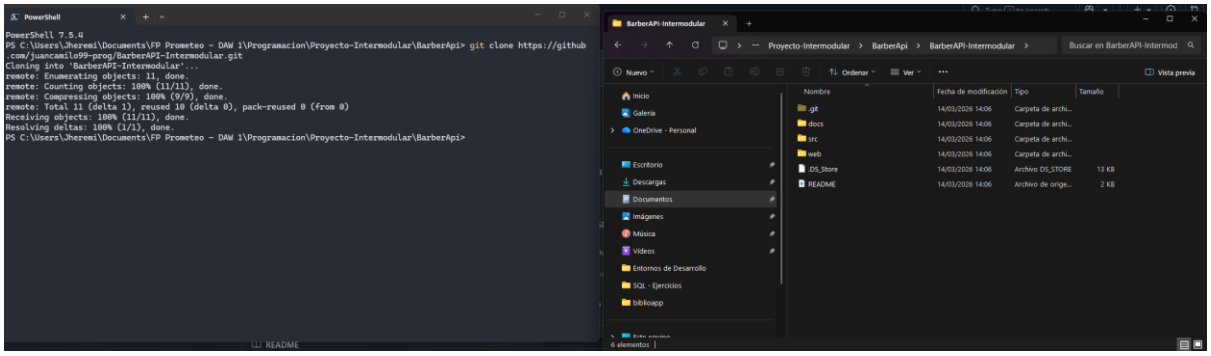
- **Flujo de la información:** La página web actúa como el punto de entrada de la información. El catálogo de servicios mostrado en el HTML refleja exactamente los datos parametrizados en la tabla `servicios` de la base de datos.
- **Simulación de la operativa transaccional:** El usuario interactúa con los formularios web para solicitar una cita. Posteriormente, el motor backend en Java es el encargado de recoger esos datos (ya sea mediante

integración directa si se utilizan Servlets/JSP, o mediante simulación en consola/interfaz de escritorio en Java) y validarlos antes de inyectarlos en la base de datos MySQL mediante sentencias preparadas (`PreparedStatement` en JDBC), evitando así vulnerabilidades como la inyección SQL.

- **Control de errores:** El Backend está programado para capturar excepciones (como fallos de conexión a la BBDD o intentos de reservar en horas ocupadas) y gestionar el flujo del programa para que no se interrumpa abruptamente.

Jheremi – Imagenes

GIT – Clone Repo – Barberi


PowerShell
PowerShell 7.5.4
PS C:\Users\Jheremi\Documents\FP Prometeo - DAW 1\Programacion\Proyecto-Intermodular\BarberApi> git clone https://github.com/juancamilo99-prog/BarberAPI-Intermodular.git
Cloning into 'BarberAPI-Intermodular'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 1), reused 10 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (1/1), done.
PS C:\Users\Jheremi\Documents\FP Prometeo - DAW 1\Programacion\Proyecto-Intermodular\BarberApi>
BarberAPI-Intermodular
Proyecto-Intermodular › BarberApi › BarberAPI-Intermodular
Buscar en BarberAPI-Intermod
Nuevo
Ordenar
Ver
Vista previa
Inicio
Galería
OneDrive - Personal
Escritorio
Descargas
Documentos
Imágenes
Música
Vídeos
Entornos de Desarrollo
SQL - Ejercicios
biblioapp
Nombre
Fecha de modificación
Tipo
Tamaño
.git
docs
src
web
.DS_Store
README
14/03/2026 14:06
Carpeta de archi...
Archivo DS_STORE
13 KB
Archivo de origen...
2 KB
6 elementos